TimesFM for GCS Enrollment Forecasting

Jason B. Horne | Prepared 2026-09-03, backtest revised 2026-09-05

What TimesFM 3.0 is, why GCS cannot deploy it, and what the data says about whether it would help.

The short version

- TimesFM 3.0 is Google's new 330M-parameter time series forecasting model. It forecasts without being trained on your data first, which is what 'zero-shot' means.
- Its weights are released under the TimesFM Non-Commercial License v1.0. Testing, evaluation, and research are allowed. Production deployment is not, and results may not be used in client deliverables or paid services.
- That rules it out for a GCS operational enrollment pipeline and rules it out entirely for a paid consulting dashboard.
- TimesFM 2.5 is the same family under Apache-2.0, with no such restriction. It is 200M parameters and supports covariates. This is the version to build on if you build anything.
- Tested properly against your own data across 187 rolling forecast origins, no version of TimesFM beat carrying yesterday's number forward. Not at 5 days out, not at 10, not at 20. The gap widens as the horizon grows.
- For the forecast you actually need, next August's opening count, every model failed by about 60 students. A one-line rule using last year's August number was off by 4.

1. What the model actually does

A traditional forecast is fit to one series. You give it Greeneville's enrollment history, it estimates parameters for Greeneville, and it predicts Greeneville. Change districts and you refit.

A time series foundation model is pretrained once on an enormous corpus of unrelated series, then forecasts a series it has never seen without any fitting step. TimesFM 3.0 was pretrained on more than 1 trillion time points. You hand it 200 days of enrollment counts and it returns the next 20, with no training run and no parameter tuning.

Three terms worth pinning down, because they are the whole pitch:

- Zero-shot. No task-specific fine-tuning. You do not train anything.
- Multivariate. TimesFM 3.0 is the first in the family pretrained to forecast several related series jointly, so the seven school-level series inform each other instead of being forecast separately.
- Covariates. Side information. Past-only covariates are things you know historically but not in advance. Past-and-future covariates are things you know in advance, such as calendar days, holidays, or a scheduled program change.

It returns 9 quantiles, the 10th through 90th percentile, so you get a prediction interval rather than a single line. For a board conversation that range matters more than the point estimate.

2. The license, in plain terms

This is the part that decides what you can do, so it goes before the fun part.

TimesFM 3.0 weights carry the TimesFM Non-Commercial License v1.0. The operative sentences:

> "Company grants you a non-exclusive, worldwide, non-transferable, non-sublicensable, revocable, royalty-free and limited license to access, use and create Derivatives of the TimesFM Model solely for your Non-Commercial Purposes."
>
> "Non-Commercial Purpose means use for testing, evaluation, or research not tied to commercial gain, production deployment, or revenue generation."
>
> "This includes internal benchmarking, academic research, and experimentation on private or public datasets, provided the results are not used in commercial decision-making, client deliverables, or paid products/services."

Applied to your actual situations:

What you want to do	TimesFM 3.0	TimesFM 2.5
Run it on your Mac to learn how it works	Yes	Yes
Benchmark it against your	Yes	Yes

enrollment history		
Put it in the nightly GCS enrollment pipeline	No, production deployment	Yes
Use its output in a TISA or budget projection	No	Yes
Ship it in a paid consulting dashboard	No, client deliverable	Yes
Show a chart from it in a conference talk	Ambiguous, ask counsel	Yes

The conference talk row is genuinely unclear. A slide is not a paid product, but the ASBO webinar in January 2027 is paid work. If you want to show TimesFM 3.0 output publicly, the clean answer is to reproduce the same chart with 2.5 and show that instead.

Note the license is revocable, and Google can offer a commercial license. If TimesFM 3.0 ever becomes worth deploying, that is the conversation to have, not a workaround.

3. What the numbers actually say

An earlier version of this document reported a single train/test split. That split was flawed: the held-out window straddled the summer break, so it blended 16 easy in-year days with 4 across-year days and measured neither. This section replaces it.

The data

269 daily PowerSchool SQL exports covering 2024-08-20 to 2026-08-12. Each file is student-level; the series is a distinct count of students with Enroll_Status = 0. Two early files use DCID rather than Students as the ID column and were mapped accordingly.

The series is not flat, which matters. Enrollment opens high in August and declines steadily all year, then resets higher the following August:

School year	Observations	First	Last	Change across the year
SY2024-2025	131	3,033	2,833	-200
SY2025-2026	134	2,946	2,794	-152
SY2026-2027	4	2,855	2,853	partial year

So there are two distinct forecasting problems here, and they behave nothing alike. Inside a school year the series drifts down slowly. Across the summer it jumps up by 60 to 113 students in a single step. They are evaluated separately below.

Problem one: forecasting inside a school year

Rolling-origin evaluation. Starting at 60 observations of history, step forward 2 observations at a time, and at each origin forecast 5, 10, and 20 school days ahead. Context never crosses a summer break. That gives 187 separate forecasts per method.

MAE is average error in students. MAPE is the same thing as a percentage.

Horizon	Method	MAPE %	MAE (students)
5 days (n=69)	Naive, carry last value	0.122	3.5
	TimesFM 2.5	0.129	3.6
	TimesFM 3.0	0.132	3.7
	Drift, 20-day slope	0.135	3.8
	Linear trend, 20-day	0.178	5.0
10 days (n=64)	Naive, carry last value	0.179	5.1
	TimesFM 2.5	0.186	5.3
	TimesFM 3.0	0.200	5.7
	Drift, 20-day slope	0.205	5.8
	Linear trend, 20-day	0.250	7.1
20 days (n=54)	Naive, carry last value	0.282	8.0
	TimesFM 2.5	0.295	8.3
	TimesFM 3.0	0.336	9.5
	Drift, 20-day slope	0.341	9.7
	Linear trend, 20-day	0.396	11.2

The ordering is identical at every horizon and it is not close to random. Carrying the last value forward wins outright. TimesFM 2.5 comes second, TimesFM 3.0 third, and the gap between naive and TimesFM 3.0 widens from 0.2 students at a one-week horizon to 1.5 students at four weeks.

Worth noting that both trend-following baselines, drift and linear trend, did worse than naive too. The within-year decline is real but it is lumpy, driven by individual withdrawals rather than a smooth slope. Any method that extrapolates the recent slope overshoots. That is a textbook random walk, and for a random walk the optimal forecast is the last value.

On the multivariate question: an earlier draft of this document said feeding the seven school-level series alongside the district total made the forecast worse. That was an artifact of the flawed split and it was wrong. Tested properly at a 20-day horizon over the same 54 origins, multivariate is slightly better than univariate, 9.2 MAE against 9.5. It still loses to naive at 8.0. The feature works as advertised, it just does not close the gap.

Problem two: forecasting across the summer

This is the forecast that actually matters after PowerSchool Predictive Enrollment was terminated. There are only two summer transitions in the data, so read this as illustration, not evidence.

Transition	Method	Predicted	Actual	Error
Spring 2025 to Aug 2025	Naive, last spring value	2,833	2,931	98
	TimesFM 2.5	2,833	2,931	98
	TimesFM 3.0	2,832	2,931	99
Spring 2026 to Aug 2026	Naive, last spring value	2,794	2,854	60
	TimesFM 2.5	2,793	2,854	60
	TimesFM 3.0	2,793	2,854	61

Every method returned essentially the spring number. None of them anticipated the August rebound, and they failed by nearly identical amounts. That is not a defect in TimesFM. The model receives a bare sequence of numbers with no dates attached. It has no way to know that a two-and-a-half month gap and a new school year sit between the last observation and the first prediction.

The rule that did work

Look at the August openings on their own, as an annual series rather than a daily one:

School year	First observation	Opening count	Year over year change
SY2024-2025	2024-08-20	3,033	
SY2025-2026	2025-08-01	2,946	-87
SY2026-2027	2026-08-09	2,855	-91

The year over year decline is remarkably steady. Applying the simplest possible rule, next August equals last August plus last year's change, predicts 2,859 against an actual 2,855. That is an error of 4 students, against roughly 60 for every model working from the daily spring series.

Two honest caveats. This is validated on exactly one held-out year, which is an anecdote. And the three opening dates are August 20, August 1, and August 9, which are different points in the early-year enrollment ramp, so the -87 and -91 figures are not strictly comparable. Both weaken the precision of the number without changing the direction of the finding.

The point is not that this rule is production ready. The point is that the right unit of analysis for your real question is the annual opening count, not the daily headcount, and a foundation model pointed at the wrong series cannot recover from that.

4. What this means

Two findings, and they lead to different places.

Inside a school year, forecasting is a solved non-problem. Errors of 3 to 8 students on a base of about 2,800 are already at the level of one family moving in or out. Nothing in this table justifies a model, a dependency, or a pipeline. If someone needs to know the headcount two weeks from now, tell them today's number and move on.

Across the summer, no method tested here works, TimesFM included, because the daily series does not contain the information. The August level is set by kindergarten intake, transfers, and out-migration over the summer, none of which appears in a sequence of spring headcounts. Feeding a bigger model the same uninformative input does not help.

The general lesson is worth more than the specific result. A foundation model can only exploit structure present in what you hand it. Handing it a daily series and asking about next August is not a modeling problem, it is a framing problem, and no amount of parameters fixes it.

There is also a procurement lesson. When the PEA replacement vendor demos an enrollment forecast, ask for mean absolute error against a naive baseline on GCS data, at a stated horizon, across multiple origins. On this evidence, any product that cannot clearly beat carrying the last number forward is selling very little.

5. Running it yourself

Environment. Anaconda Python 3.12 at /opt/anaconda3/bin/python3, torch 2.10 with MPS available. All results here were computed on CPU.

```
/opt/anaconda3/bin/python3 -m pip install 'timesfm[torch]'
```

Two install notes. It upgrades protobuf to 5.x, which breaks the streamlit 1.32 pin already in your Anaconda environment. Nothing else appeared to break, but if you use streamlit, install TimesFM in a separate conda environment. Second, the README example on GitHub does not match the shipped API. The real entry points are:

```
import timesfm
timesfm.TimesFM_2p5_200M_torch   # Apache-2.0, deployable
timesfm.TimesFM3Forecaster      # non-commercial only
timesfm.ForecastConfig
```

TimesFM 2.5, the version you can actually use:

```
import numpy as np, pandas as pd, timesfm
from timesfm import ForecastConfig

df  = pd.read_csv("enrollment_series.csv", parse_dates=["date"])
ctx = df["total"].astype(np.float32).values[:-20]

m = timesfm.TimesFM_2p5_200M_torch.from_pretrained(
        "google/timesfm-2.5-200m-pytorch", torch_compile=False)
m.compile(ForecastConfig(max_context=256, max_horizon=20,
```

```
                                normalize_inputs=True,
                                use_continuous_quantile_head=True,
                                fix_quantile_crossing=True))

    point, quantiles = m.forecast(horizon=20, inputs=[ctx])   # inputs is a list
    print(point[0][:20])
    print(np.asarray(quantiles).shape)    # (1, 20, 10)
```

Set torch_compile=False. Compiling takes several minutes on CPU and buys nothing for a one-off forecast.

TimesFM 3.0, evaluation only:

```
    import numpy as np, timesfm

    f3 = timesfm.TimesFM3Forecaster.from_pretrained(
            "google/timesfm-3.0-pytorch", device="cpu")

    # univariate: context is (1, n)
    out = f3.predict(context=ctx.reshape(1, -1), horizon=20)
    print(out.forecast.shape)             # (1, 20)

    # multivariate: one row per series, row 0 is the target
    cols  = ["total","sch_EV","sch_GHS","sch_GMS",
             "sch_HH","sch_HI","sch_TOPS-G","sch_TV"]
    panel = df[cols].astype(np.float32).values[:-20].T   # (8, n)
    out2  = f3.predict(context=panel, horizon=20)
    print(out2.forecast[0])               # forecast for 'total'

    # batching many origins at once:
    for o in f3.predict_batch(contexts=[c1, c2, c3], horizon=20):
        print(o.forecast)

    # ForecastOutput fields are: forecast, quantiles, ts_id
    # (there is no .mean_forecast, despite what some examples show)
```

The 3.0 checkpoint is roughly 1.2 GB on disk, 2.5 is 882 MB. Neither is a gated repository, so both download without a Hugging Face login.

6. Covariates, which is where this gets useful

Both versions accept side information, and this is the part worth your time if you revisit the problem. TimesFM 2.5 exposes forecast_with_covariates with four kinds:

- dynamic_numerical_covariates: varies over time, numeric. Days since term start.
- dynamic_categorical_covariates: varies over time, categorical. In session or not, testing window, month.
- static_numerical_covariates: fixed per series. Prior year October 1 count.
- static_categorical_covariates: fixed per series. School, grade band, Title I status.

For enrollment the obvious wins are a school-calendar flag as a past-and-future covariate, since you know the calendar in advance, and grade plus school as static categoricals so one call forecasts every school-grade cell at once rather than looping over 91 separate models.

Given the results in section 3, treat this as the thing to try if you take another run at the annual problem, not as a reason to revisit the daily one.

7. What I would actually do

- Do not build a daily enrollment forecaster. Naive won at every horizon over 187 origins. Carrying the last count forward is the correct model, and it costs nothing to maintain.
- Do not put TimesFM 3.0 anywhere near a GCS pipeline or a consulting deliverable regardless of accuracy. The license prohibits production use and bars results from paid services.
- For the post-PEA August projection, work the annual series, not the daily one. Build August opening counts by grade and school going back five or more years from the ADM workbooks, then test the year over year rule against anything fancier. On two years of data the trivial rule was 15 times more accurate than a 330M-parameter model.
- Check whether the Woolpert 2024 long range facilities plan already contains enrollment projections. If it does, that is a five-year professional projection you have already paid for, and it should be the baseline any new work has to beat.
- If you do revisit modeling, use TimesFM 2.5 under Apache-2.0 with school and grade as static categoricals. Same research lineage, no license problem.
- Hold the eventual PEA replacement vendor to the standard in section 4: MAE against a naive baseline, on GCS data, at a stated horizon, across multiple origins.

Sources

- Google Research, TimesFM-3: A zero-shot foundation model for multivariate forecasting. research.google/blog/timesfm-3-a-zero-shot-foundation-model-for-multivariate-forecasting (330M parameters, >1T time points, 9 quantiles, benchmark rankings)
- Hugging Face, google/timesfm-3.0-pytorch, LICENSE file. TimesFM Non-Commercial License v1.0. All quoted license text.
- Hugging Face, google/timesfm-2.5-200m-pytorch. License apache-2.0, 200M parameters.
- GitHub, google-research/timesfm. Per-version license table and pip package.
- Google Cloud, The AI.FORECAST function and the TimesFM forecasting tutorial. BigQuery serves TimesFM 2.0 (default) and 2.5.
- GCS daily PowerSchool SQL exports, 269 files, OneDrive-GCS/Python/Enrollment Data. All accuracy figures computed locally 2026-09-05 by backtest2.py and backtest3.py in this folder.

Caveats on this document

- The across-summer comparison rests on two transitions and the year over year rule on one held-out year. Neither is evidence in any statistical sense. The in-year rolling-origin results, at 187 forecasts, are solid.
- The three August opening dates are August 20, August 1, and August 9, which are different points in the early-year enrollment ramp. The -87 and -91 year over year figures are therefore not strictly comparable.
- Rolling-origin contexts never cross a summer break, so within-year results are clean. No date or calendar features were supplied to any model; all were given a bare sequence. Supplying calendar covariates could change the in-year ranking, though given naive already wins, there is limited room.
- The daily series ends 2026-08-12. Either the daily pull has stopped or recent files live elsewhere. Worth checking before relying on the pipeline.
- Benchmark claims about rank on GIFT-Eval, fev-bench, and TIME are Google's own, reported in its release blog, and are not independently verified here.